

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт

Про виконання лабораторної роботи №4
«ОСНОВНІ ТИПИ ЗМІННИХ. ОПЕРАЦІЇ ТА ЇХ ПРІОРИТЕТ.»

Виконав:

Студент групи ФЕП-14с

Лаврентій Остап Романович

Перевірив:

Асистент **Васюта В. М.**

Львів 2026

Мета: Вивчити поняття: основні типи змінних, вказівники, операції та їх пріоритет.

Хід роботи:

1. Написав програму, яка виводить результат кожного оператора змінних a і b

```
void main(){
    int a = 9;
    int b = 7;

    printf("a=%d, b=%d \n", a, b);

    printf("a + b = %d\n", a + b);
    printf("a - b = %d\n", a - b);
    printf("a * b = %d\n", a * b);
    printf("a / b = %d\n", a / b);
    printf("a %% b = %d\n", a % b);

    printf("a == b: %d\n", a == b);
    printf("a != b: %d\n", a != b);
    printf("a > b: %d\n", a > b);
    printf("a < b: %d\n", a < b);
    printf("a >= b: %d\n", a >= b);
    printf("a <= b: %d\n", a <= b);

    printf("(a > 0) && (b > 0): %d\n", (a > 0) && (b > 0));
    printf("(a > 0) || (b < 0): %d\n", (a > 0) || (b < 0));
    printf("!(a > 0): %d\n", !(a > 0));
```

Також для побітових операцій з виведенням у бінарному вигляді

```
printf("a = ");
printBinary(a);
printf("\n");

printf("b = ");
printBinary(b);
printf("\n");

printf("a & b = ");
printBinary(a & b);
printf(" (%d)\n", a & b);

printf("a | b = ");
printBinary(a | b);
printf(" (%d)\n", a | b);

printf("a ^ b = ");
printBinary(a ^ b);
printf(" (%d)\n", a ^ b);

printf("~a = ");
printBinary(~a);
printf(" (%d)\n", ~a);

printf("a << 1 = ");
printBinary(a << 1);
printf(" (%d)\n", a << 1);

printf("a >> 1 = ");
printBinary(a >> 1);
printf(" (%d)\n", a >> 1);
```

```
void printBinary(int number) {
    unsigned int num = (unsigned int)number;

    for (int i = 31; i >= 0; i--) {
        printf("%d", (num >> i) & 1);
    }
}
```

[illegible]

2. Написав щоб програма приймала число, виводила його посилання і виводила його через вказівник

```
int enteredNumber;
int* enteredNumberPointer;
int***** pointerToThePowerOfTen;

printf("\nEnter a number: ");
scanf("%d", &enteredNumber);
enteredNumberPointer = &enteredNumber;
printf("\nNumber: %d", enteredNumber);
printf("\nNumber address: %p", enteredNumberPointer);
printf("\nNumber value from pointer: %d", *enteredNumberPointer);
```

Результат:

```
Enter a number: 125

Number: 125
Number address: 000000124D0FFA70
Number value from pointer: 125
PS C:\Users\Dell\Documents\лаб\роботи\ ОП\Env\lab4>
```

3. Написав програму, яка приймає коефіцієнти для квадратного рівняння і розв'язує його виводячи корені

```
printf("\n\nQuadratic equation\n\n");
float Ax, Bx, Cx, D, x1, x2;
while (1 == 1)
{
    printf(" Enter A: ");
    scanf("%f", &Ax);
    printf(" Enter B: ");
    scanf("%f", &Bx);
    printf(" Enter C: ");
    scanf("%f", &Cx);

    printf("\n");

    if (Ax == 0)
    {
        printf("This isnt a quadratic equation\n");

        if (Bx != 0)
        {
            float x0 = -Cx / Bx;
            printf("Linear equation root: x = %f\n", x0);
        }
        else if (Cx == 0)
        {
            printf("Infinite solutions\n");
        }
        else
        {
            printf("No solutions\n");
        }
    }
    else
    {
        D = Bx*Bx - 4*Ax*Cx;
        printf("Discriminant = %f\n", D);
    }
}
```

Знаходить корені через дискримінант

```
if (D > 0)
{
    x1 = (-Bx + sqrt(D)) / (2 * Ax);
    x2 = (-Bx - sqrt(D)) / (2 * Ax);

    printf("x1 = %f\n", x1);
    printf("x2 = %f\n", x2);
}
else if (D == 0)
{
    float x0 = -Bx / (2 * Ax);

    printf("x = %f\n", x0);
}
else
{
    printf("No roots\n");
}
}
printf("\n");
```

Результат:

```
Quadratic equation

Enter A: 1
Enter B: 2
Enter C: 3

Discriminant = -8.000000
No roots

Enter A: 5
Enter B: 0
Enter C: 3

Discriminant = -60.000000
No roots

Enter A: 0
Enter B: 0
Enter C: 0

This isnt a quadratic equation
Infinite solutions

Enter A: 1
Enter B: -5
Enter C: 6

Discriminant = 1.000000
x1 = -125.500000
x2 = 130.500000

Enter A: 
```

Висновок: Було випробувано усі арифметичні, реляційні і побітові оператори мови C. Для побітових операторів виведення бінарної форми у термінал способом остачі ділення на 2 більше не працювало, адже перший біт `int` відповідає за те, чи додатне число, чи від'ємне. Коли було перероблено функцію `printBinary` щоб вона брала біти напряму, стало видно усі 32 біти типу `int`.

Для того, щоб квадратне рівняння можна було виконувати багато разів без рестарту програми, було поміщено логіку в нескінченний цикл і раз C не дає використовувати `while(true)` було використано `while(1==1)` з вічним оператором